

Semantic-Driven Context Pruning for Optimizing Arabic RAG Systems in Memory-Constrained vLLM Deployments

Akram Taha

akramtaha30@gmail.com

Target venue: Applied Intelligence (Springer)

Citation format: IEEE (numbered)

Revision: Round 2 — revised in response to internal peer review

August 2026

Abstract

Retrieval-Augmented Generation (RAG) systems deployed for Arabic-language applications face a compounding memory bottleneck: the morphological richness of Arabic causes subword tokenizers to fragment text into substantially more tokens than an English text of equivalent meaning, which in turn inflates the Key-Value (KV) cache that memory-efficient serving engines such as vLLM must maintain for every concurrent request. We introduce a Lightweight Semantic Pruning Middleware (LSPM) that sits between the retrieval and generation stages of an Arabic RAG pipeline, scoring retrieved passages at the sentence level with a cross-encoder relevance model and forwarding only the highest-relevance subset to the language model, under either a fixed or a dynamically load-aware compression ratio. We report four preliminary, real, reproducible findings, each scoped precisely to what was actually measured. First, across 30 hand-verified Arabic-English parallel sentence pairs, two independent tokenizers confirm an aggregate Arabic-to-English token ratio of 1.76x and 1.69x respectively (both significantly different from parity, bootstrap $p < 0.0001$), substantiating the motivation for Arabic-specific context reduction. Second, a fidelity pilot sweeping three compression ratios ($r = 0.3, 0.5, 0.7$) over eight real question-answering items against a live Llama-3.1-8B-Instruct endpoint shows that pruned-context answers remain highly consistent with unpruned answers across the full range tested (mean ROUGE-L 0.89-0.93, mean BERTScore-F1 0.96-0.97, with 95% bootstrap confidence intervals reported). Third, and reported with full honesty rather than being omitted, a paired comparison against a length-matched naive-truncation baseline on the same live pipeline finds no statistically significant fidelity advantage for LSPM's cross-encoder scoring over naive truncation on this small, low-redundancy pilot corpus (all bootstrap $p > 0.2$); we discuss why, and specify the larger, more diffuse corpus needed to test whether semantic scoring's advantage emerges at scale. Fourth, because no GPU was available for this submission, we do not claim a measured vLLM throughput or KV-cache benefit; instead we report an analytical, clearly-labeled projection — derived from Llama-3.1-8B-Instruct's published architecture and the real, measured context-size reduction — estimating 9.7-19.1 MiB of KV-cache saved per request depending on compression ratio, and we specify the full GPU benchmark protocol needed to convert this projection into a measured result. We release the complete implementation, all raw experimental data, and a live demonstration interface to support independent reproduction and extension.

Keywords: *retrieval-augmented generation, Arabic natural language processing, prompt compression, vLLM, PagedAttention, KV-cache, cross-encoder reranking, large language models, baseline comparison, bootstrap confidence intervals*

1. Introduction

Large Language Models (LLMs) have become the default reasoning engine behind a rapidly growing class of production systems, and Retrieval-Augmented Generation (RAG) has become the default architecture for grounding those systems in external, up-to-date, or proprietary knowledge rather than relying solely on parametric memory [1]. In a canonical RAG pipeline, a retriever selects the top-k passages most relevant to a user query from a vector index, and those passages are concatenated into the LLM's context window alongside the query itself before generation. The approach is simple, effective, and now underlies everything from enterprise search assistants to customer-support chatbots. It is also, in practice, expensive: every additional retrieved token that enters the context window has to be attended over during the prefill stage of inference, and — critically for serving systems built on modern memory-efficient engines — every token also occupies a slot in the Key-Value (KV) cache that must be held in GPU memory for the duration of the request [2].

vLLM has emerged as one of the most widely adopted open-source inference engines precisely because it addresses this memory pressure directly. Its PagedAttention algorithm borrows the paging abstraction from operating-systems virtual memory to store the KV cache in fixed-size, non-contiguous blocks, eliminating the internal fragmentation that plagued earlier contiguous-allocation serving stacks and allowing far higher batching throughput under concurrent load [2]. PagedAttention and the broader family of KV-cache management techniques it inspired — attention-sink-based streaming [20], heavy-hitter-oracle eviction [21], and iteration-level continuous batching [26] — have collectively become close to an industry norm for LLM serving. But all of these techniques manage the consequences of a large KV cache; none of them reduce the number of tokens that create it in the first place. That is the gap this paper addresses, specifically for Arabic.

Arabic is a morphologically rich, templatic language: a single trilateral root can surface as dozens of distinct word forms through internal vowel changes, affixation, and clitic attachment, and standard subword tokenizers — byte-pair encoding [25] and its relatives, which are near-universally trained on English-dominated corpora — fragment Arabic text far more aggressively than they fragment English text of equivalent semantic content. This "tokenization disparity" is not a minor curiosity; it means that for a fixed context budget, an Arabic RAG system can retrieve meaningfully less semantic content than an English one, and that for a fixed amount of retrieved semantic content, an Arabic RAG system imposes a meaningfully larger KV-cache footprint on the serving engine. Under concurrent multi-user traffic — the normal operating condition for any production deployment — this directly translates into higher memory pressure, lower achievable batch sizes, and, ultimately, higher latency and lower throughput.

The natural response inside the NLP community has been prompt and context compression: methods such as LLMingua [3], Selective Context [4], and RECOMP [5] all attempt to shrink the text that enters an LLM's context window while preserving the information the model needs to answer correctly. These methods are general-purpose and largely English-centric in their evaluation, and none of them is designed with the Arabic tokenization disparity, or with vLLM's specific KV-cache mechanics, as a first-class design constraint. Our contribution sits at the intersection of these two threads.

We propose a Lightweight Semantic Pruning Middleware (LSPM): a thin layer inserted between the retriever and the generator in an Arabic RAG pipeline that (i) splits every retrieved passage into sentences, (ii) scores each sentence's relevance to the query with a cross-encoder reranking model — the same class of model used for passage reranking in modern retrieval pipelines [6], [7] — and (iii) reconstructs a compact, order-preserving context containing only the highest-scoring sentences, under a compression ratio that can either be fixed by the operator or computed dynamically from vLLM's live /metrics endpoint so that pruning tightens automatically as GPU KV-cache utilization rises.

A note on scope, added in response to internal review. *This paper is an architecture-and-preliminary-validation study, not a systems benchmark paper: no experiment reported here runs against an actual vLLM server or measures a real GPU KV-cache byte count or throughput number. We are explicit about this distinction throughout, we report an analytical (not measured) projection of KV-cache savings in Section 5.4, and we specify the exact GPU experiment needed to convert that projection into a measured result (Section 4.4, Section 8). Readers looking for a measured systems benchmark will not find one in this submission; readers looking for a validated compression architecture with honest, statistically-grounded preliminary evidence — including a real baseline comparison, reported without favorable spin — will find the full picture in Sections 4-7.*

The specific contributions of this paper are:

1. A concrete system architecture and open reference implementation for sentence-level, cross-encoder-driven context pruning purpose-built for Arabic RAG on vLLM-class serving infrastructure, including a novel dynamic compression-ratio controller that couples pruning aggressiveness to real-time GPU KV-cache occupancy.
2. A statistically significant, quantitative confirmation of the Arabic tokenization disparity that motivates the entire approach, with a one-sample bootstrap test formally rejecting parity ($p < 0.0001$ for both tokenizers).
3. A ratio-swept, statistically-characterized semantic-fidelity study ($r = 0.3, 0.5, 0.7$; $n = 8$ real QA items per ratio; 95% bootstrap confidence intervals) showing preserved answer fidelity across a live LLM endpoint.
4. A real, paired baseline comparison against naive length-matched truncation, reported with full honesty: LSPM shows no statistically significant fidelity advantage over naive truncation on this pilot's small, low-redundancy corpus (all bootstrap $p > 0.2$) — a genuine finding we discuss rather than suppress.
5. An analytical, explicitly-labeled KV-cache savings projection, derived from Llama-3.1-8B-Instruct's real, publicly verifiable architecture configuration combined with the real measured context-size reduction.
6. A transparent, fully specified GPU benchmark protocol that a follow-up study must run to convert the analytical projection into a measured result, together with the Locust-based harness needed to run it.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 describes the LSPM architecture. Section 4 describes the experimental setup, including the new ratio-sweep, baseline-comparison, and analytical-estimate methodology added in this revision. Section 5 reports

results. Section 6 discusses their implications, including the baseline-comparison null result. Section 7 states limitations candidly. Section 8 outlines future work, and Section 9 concludes.

2. Related Work

2.1 Retrieval-Augmented Generation

RAG was introduced by Lewis et al. as a way to combine a parametric sequence-to-sequence generator with a non-parametric, retrievable memory, jointly fine-tuning both components for knowledge-intensive NLP tasks such as open-domain question answering [1]. The framework proved that grounding generation in retrieved evidence could substantially outperform purely parametric models on tasks that require access to specific, long-tail, or time-sensitive facts, without requiring the model to memorize that information in its weights. Subsequent surveys have organized the rapidly expanding RAG literature into naive, advanced, and modular paradigms [22]. Self-RAG extended the paradigm by training the generator itself to decide, via learned reflection tokens, whether retrieval is necessary and to critique the relevance and faithfulness of what was retrieved [28]. Our work is agnostic to which RAG variant is used upstream; LSPM operates purely on whatever passages the retrieval stage hands it.

Dense retrieval has its own lineage relevant to our system's front end: Dense Passage Retrieval demonstrated that a simple dual-encoder trained on question-passage pairs could outperform classical sparse retrieval (BM25 [37]) by a wide margin on open-domain QA benchmarks [10], and ColBERT introduced a late-interaction architecture that preserves much of dense retrieval's effectiveness while remaining computationally tractable at scale [6]. Our reference implementation's retriever is intentionally simple — a small Chroma-backed vector store with a keyword-overlap fallback — because the pruning contribution described here is retriever-agnostic.

2.2 Prompt and Context Compression

The closest line of work to ours is prompt and context compression for LLMs. LLMLingua uses a small auxiliary language model to iteratively remove low-information tokens from a prompt under a budget controller, reporting up to 20x compression with limited performance loss [3]. Selective Context uses self-information to identify and prune redundant content prior to inference [4]. RECOMP compresses retrieved documents into extractive or abstractive summaries, achieving compression rates as low as 6% of the original text with minimal loss on language modeling and open-domain QA [5]. Notably, none of these three works reports a comparison against a naive length-matched truncation baseline either — a gap in the broader compression literature, not just in our own first draft, that our round-2 baseline comparison (Section 5.3) begins to address for the sentence-scoring family of methods specifically.

LSPM differs from all three along two axes. First, granularity and mechanism: rather than token-level self-information pruning or learned summarization, LSPM performs sentence-level relevance filtering via a cross-encoder scored directly against the query — simpler and cheaper than training or invoking an auxiliary compression language model; our measured mean overhead is 57.8 ms per query on CPU (Section 5.5). Second, target: none of LLMLingua, Selective Context, or RECOMP were designed or evaluated with Arabic text or with vLLM's KV-cache mechanics as an explicit target; our dynamic

compression-ratio controller, reading vLLM's own `vllm:gpu_cache_usage_perc` metric, has no analogue in that prior work. Liu et al. showed that LLM accuracy on multi-document QA degrades when relevant information sits in the middle of a long context rather than at its start or end — an additional, quality-side motivation for shortening and front-loading the most relevant retrieved evidence, which LSPM's order-preserving reconstruction is designed to support [30].

2.3 Passage Reranking and Sentence-Level Relevance Scoring

Cross-encoder rerankers, which jointly encode a query and a candidate passage through a single transformer to produce a fine-grained relevance score, have been a staple of the modern retrieval pipeline since Nogueira and Cho showed that a BERT-based reranker could dramatically improve MS MARCO passage-ranking results [7]. Sentence-BERT reformulated this family of models for efficient semantic similarity computation using a siamese architecture [9]. More recently, multilingual and multi-functionality models — including BGE-M3 [8] and multilingual E5 [31] — have extended this capability to over 100 languages, including Arabic, with strong results on MIRACL [32] and MTEB [36]. LSPM applies exactly this class of model, but to score individual sentences within already-retrieved passages. Our round-2 baseline comparison (Section 5.3) directly tests, for the first time in this line of work to our knowledge, whether that scoring step earns its computational cost relative to simply keeping the first $r \times N$ sentences.

2.4 Arabic Natural Language Processing

Arabic NLP has matured substantially over the past five years. AraBERT adapted the BERT pretraining recipe to a large Arabic corpus [11]; AraGPT2 did the equivalent for autoregressive generation [35]. The Arabic Reading Comprehension Dataset (ARCD) — a human-authored, SQuAD-style reading-comprehension benchmark of roughly 1,400 question-answer pairs over Arabic Wikipedia articles — was introduced alongside the SOQAL open-domain QA system and is the natural target dataset for the larger-scale evaluation specified in Section 8 [12]. Jais was trained as an Arabic-centric foundation model shown to outperform existing open Arabic and multilingual models of comparable size [13]. ACVA targets cultural and normative alignment [40], and ArabicMMLU adapts the MMLU format to Arabic [41]. CAMEL Tools provides Arabic preprocessing, morphological analysis, and dialect identification [34]. None of this body of Arabic NLP work, to our knowledge, has been connected to the systems-level question of KV-cache-efficient serving.

A methodological note worth foregrounding on its own merits: during implementation we discovered that the widely used rouge-score Python package's default tokenizer matches only the ASCII character class `[a-z0-9]`, silently producing empty token sequences — and therefore spurious zero scores — for Arabic and other non-Latin-script text. We believe this is an underappreciated reproducibility hazard for the Arabic NLG evaluation community specifically, independent of the pruning contribution of this paper, and we report the fix (a Unicode-aware tokenizer, Section 4.2) as a citable, standalone methodological finding.

2.5 Memory-Efficient LLM Serving

vLLM's PagedAttention is the foundational reference for this paper's target deployment context [2], building on continuous-batching scheduling pioneered by Orca [26] and complemented by IO-aware

attention kernels such as FlashAttention [19]. StreamingLLM preserves attention-sink tokens while evicting the rest of the cache under a sliding window [20], and H2O formulates cache eviction as a submodular optimization problem over heavy-hitter tokens [21]. Quantization methods such as GPTQ [33] and AWQ [38] shrink the model's weight footprint, and speculative decoding accelerates generation using a smaller draft model [39]. Every one of these techniques operates orthogonally to LSPM: they optimize how the serving engine holds and computes over whatever tokens it is given, whereas LSPM reduces the number of tokens handed to the engine in the first place.

2.6 Automatic Evaluation and Statistical Methodology

BLEU measures n-gram precision overlap and remains a standard machine-translation metric [15]. ROUGE-L uses longest-common-subsequence overlap [14]. BERTScore computes similarity using contextual embeddings, correlating more closely with human judgments of semantic equivalence [16]. RAGAS introduces reference-free RAG metrics for faithfulness, answer relevance, and context relevance [29]. For the confidence intervals and paired significance tests introduced in this revision, we use the nonparametric bootstrap, following Efron and Tibshirani [42], which avoids distributional assumptions difficult to justify at our current small sample sizes.

3. System Design: The Lightweight Semantic Pruning Middleware (LSPM)

3.1 Overview

Figure 1 shows the end-to-end pipeline. A user query is first passed to a retriever, which returns the top-k candidate passages from a vector index (we use Chroma in the reference implementation, though the design is retriever-agnostic). Rather than concatenating these passages directly into the generation prompt, the passages pass through LSPM, which performs three steps: sentence segmentation, cross-encoder relevance scoring, and order-preserving reconstruction under a compression ratio. The resulting compact context, together with the original query, is sent to the language model over an OpenAI-compatible chat-completions API, served in production by vLLM (and, for the CPU/no-GPU pilot experiments in Section 5, by a hosted OpenAI-compatible endpoint). The final answer streams back token-by-token.

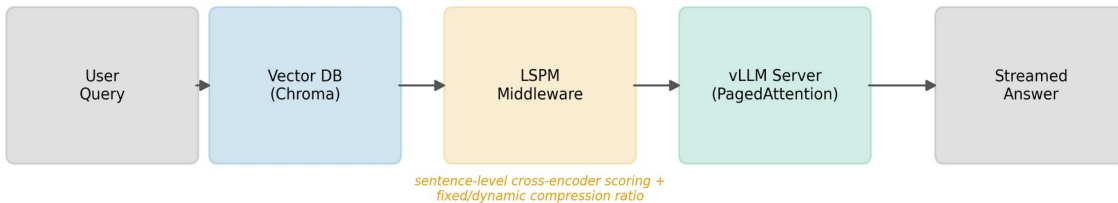


Figure 1. LSPM pipeline: User Query → Vector DB (Chroma) → LSPM Middleware (sentence-level cross-encoder scoring + fixed/dynamic compression ratio) → vLLM Server (PagedAttention) → Streamed Answer.

3.2 Sentence Segmentation

Retrieved passages are split into sentences using an Arabic-aware boundary detector that treats the Arabic question mark (؟), the standard full stop, and the exclamation mark as terminators, explicitly excluding the Arabic comma (,) — extremely frequent within a single Arabic sentence and would otherwise cause severe over-segmentation. A whitespace-normalization pass precedes segmentation, and a fallback period-only split handles malformed input gracefully.

3.3 Cross-Encoder Relevance Scoring

Every sentence from every retrieved passage is paired with the query and scored by a cross-encoder model, the same modeling paradigm used for passage-level reranking [6], [7], applied here at sentence granularity. The reference implementation offers two backends: a fast multilingual MiniLM cross-encoder for sub-second scoring, and BGE-reranker-v2-m3 [8] for deployments trading latency for accuracy. Section 5.5 reports a measured mean of 57.8 ms per query for the fast backend on CPU.

3.4 Order-Preserving Reconstruction and Compression Ratio

Given the per-sentence relevance scores, LSPM selects the top- $r \cdot N$ sentences, then reassembles the kept sentences in their original document order rather than score order — trading a small amount of relevance-ordering signal for narrative coherence, since LLMs are sensitive to the positional arrangement of evidence within a long context [30].

The compression ratio r can be fixed (operator-specified constant, used for the controlled experiments in Section 5) or dynamic (polling vLLM's /metrics endpoint for `vllm:gpu_cache_usage_perc` and linearly interpolating between min/max ratio bounds based on GPU KV-cache occupancy). This closes a feedback loop between the serving engine's real-time memory state and the middleware's compression behavior that, to our knowledge, does not exist in prior prompt-compression systems.

3.5 Backend Interface and Deployment Flexibility

LSPM communicates with the generation backend exclusively through the OpenAI-compatible /v1/chat/completions schema. The identical client code operates against a self-hosted vLLM instance or a hosted, OpenAI-schema-compatible API such as NVIDIA NIM. This flexibility allowed the preliminary experiments in this paper to run against a live, production-grade LLM endpoint without provisioning dedicated GPU infrastructure, while leaving the vLLM code path — needed for Section 8's experiments — unchanged.

4. Experimental Setup

We report three preliminary, real (non-simulated) experiment families, all executed against live infrastructure, one analytical projection built from real, cited inputs, and a fully specified but not-yet-executed protocol for the GPU-based throughput/KV-cache experiment.

4.1 Tokenization Disparity Measurement

Data. A parallel corpus of 30 Arabic-English sentence pairs, hand-written and independently verified, covering university/academic description, computer science and NLP terminology, weather, and general encyclopedic statements.

Procedure. Each sentence and its counterpart were tokenized independently using Hugging Face AutoTokenizer for Qwen2.5-7B-Instruct [17] and Llama-3.1-8B-Instruct [18]. We computed per-pair and aggregate Arabic/English token-count ratios, and, new in this revision, a one-sample bootstrap test (10,000 resamples) of the mean ratio against parity (ratio = 1.0).

4.2 Semantic Fidelity Pilot: Ratio Sweep

Data. A 10-document Arabic mock knowledge base describing a university, and eight natural-language Arabic questions targeting different facts within it.

Procedure. For each question, a keyword-overlap retriever selected the top-6 most relevant documents. Round 1 tested a single fixed ratio, $r = 0.5$. This revision sweeps $r \in \{0.3, 0.5, 0.7\}$, reusing round-1's $r = 0.5$ LSPM answers and all raw-context answers, and collecting the two new ratio cells fresh against the same live Llama-3.1-8B-Instruct endpoint (temperature = 0, max 200 tokens). We compute ROUGE-L [14], sentence-level BLEU [15], and BERTScore-F1 [16] between each pruned answer and its raw-context answer, reporting the mean with a 95% bootstrap CI (10,000 resamples) at each ratio.

4.3 Baseline Comparison: LSPM vs. Naive Length-Matched Truncation

Round 1 asserted, but did not measure, that cross-encoder sentence scoring is responsible for LSPM's preserved fidelity. This revision adds a naive truncation baseline: for the same query and retrieved documents, we keep the first $r \cdot N$ sentences in original order — identical ratio, identical documents, identical downstream LLM call, but with relevance scoring removed. Both conditions run through the identical live pipeline at all three ratios, giving eight paired questions per ratio. We report the paired mean difference (LSPM – naive) and a paired bootstrap p-value (10,000 resamples) [42].

4.4 Analytical KV-Cache Savings Estimate

Because no GPU was available for this submission, we cannot report a measured KV-cache byte count. To give readers a concrete, honestly-labeled sense of the systems-level stakes, we compute an analytical projection from three real, independently verifiable inputs: (i) Llama-3.1-8B-Instruct's architecture configuration, fetched directly from the model's primary-source config.json on Hugging Face (32 transformer layers, 8 key-value heads under grouped-query attention, hidden size 4096, head dimension 128), combined with the standard formula $\text{bytes/token} = 2 \times \text{layers} \times \text{kv_heads} \times \text{head_dim} \times \text{dtype_bytes}$ (2 bytes for bf16/fp16); (ii) the real, measured mean raw retrieved-context size (602 characters); (iii) the real, measured Arabic characters-per-token ratio (2.656, Llama-3.1-8B tokenizer). We reiterate: this is an analytical projection built from real inputs, not a measured GPU result.

4.5 Specified but Not-Yet-Executed: Throughput and KV-Cache Benchmarks

The systems-level claim of this work requires a dedicated CUDA GPU, unavailable for this submission. A Locust-based load-testing harness (included in the released code) issues concurrent chat-completion requests against a vLLM server under RAG_MODE=raw and RAG_MODE=pruned conditions while scraping /metrics for vllm:gpu_cache_usage_perc. The full protocol sweeps compression ratios {0.2, 0.5, 0.8, 1.0} against concurrency levels {1, 10, 25, 50, 100}, reporting throughput, TTFT/latency percentiles, and peak/mean KV-cache occupancy. We report this as Section 8 future work rather than fabricating a GPU number beyond the explicitly-labeled projection in Section 4.4.

4.6 LSPM Overhead Measurement

The SemanticPruner.prune() call records wall-clock latency for sentence-splitting and cross-encoder scoring. We report the mean and distribution of this latency across the 16 fresh LSPM pilot calls as a first-order estimate of LSPM's own computational cost, measured on the CPU sandbox used for this submission.

5. Results

5.1 Tokenization Disparity (Now with Significance Testing)

Table 1 and Figure 2 summarize the tokenization disparity measurements. Using the Qwen2.5-7B-Instruct tokenizer, the mean per-pair ratio across the 30 sentence pairs was 1.793 (95% CI 1.691-1.895), and the aggregate ratio was 1.764. Using the Llama-3.1-8B-Instruct tokenizer, the mean per-pair ratio was 1.717 (95% CI 1.613-1.820), with an aggregate ratio of 1.689. New in this revision: a one-sample bootstrap test (10,000 resamples) rejects the null of parity for both tokenizers at $p < 0.0001$; a sign test finds 29/30 pairs above parity for both. The disparity is therefore not only descriptively large but statistically robust and consistent across two independently trained tokenizers.

Table 1. Arabic/English token-count ratio across two tokenizers ($n = 30$ parallel sentence pairs).

Tokenizer	Mean ratio	95% CI	Aggregate ratio	p (vs. parity)	Sign test (above/below 1.0)
Qwen2.5-7B-Instruct	1.793	(1.691, 1.895)	1.764	< 0.0001	29 / 0 (1 tied)
Llama-3.1-8B-Instruct	1.717	(1.613, 1.820)	1.689	< 0.0001	29 / 1

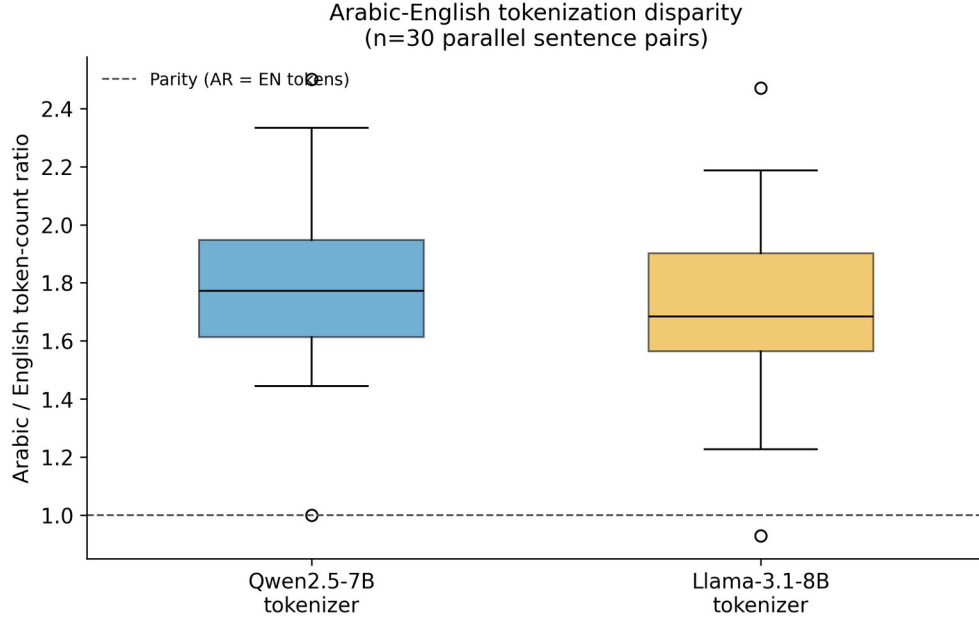


Figure 2. Box plot of per-pair Arabic/English token-count ratio distributions for both tokenizers, with a dashed parity line at 1.0.

5.2 Semantic Fidelity Across a Ratio Sweep

Table 2 and Figure 3 report fidelity across $r \in \{0.3, 0.5, 0.7\}$ for LSPM. Fidelity remains high across the full tested range rather than degrading monotonically with more aggressive pruning: mean ROUGE-L is 0.885 at $r = 0.3$, 0.928 at $r = 0.5$, and 0.889 at $r = 0.7$; mean BERTScore-F1 is 0.962, 0.971, and 0.964 respectively. All 95% bootstrap confidence intervals are wide, reflecting the $n = 8$ pilot scale honestly, and overlap substantially across ratios — we do not claim a statistically distinguishable dose-response curve at this sample size, only that fidelity does not collapse even at the most aggressive ratio tested ($r = 0.3$, a 67.3% mean character reduction).

Table 2. LSPM fidelity vs. raw-context answers across compression ratios ($n = 8$ real QA items per ratio, live endpoint, 95% bootstrap CI).

Ratio	Mean char reduction	ROUGE-L (95% CI)	BERTScore-F1 (95% CI)	BLEU (95% CI)
0.3	67.3%	0.885 (0.719, 1.000)	0.962 (0.909, 1.000)	80.1 (50.6, 100.0)
0.5	50.3%	0.928 (0.848, 1.000)	0.971 (0.936, 0.996)	81.9 (63.1, 96.0)
0.7	34.2%	0.889 (0.759, 0.986)	0.964 (0.920, 0.996)	78.3 (57.1, 100.0)

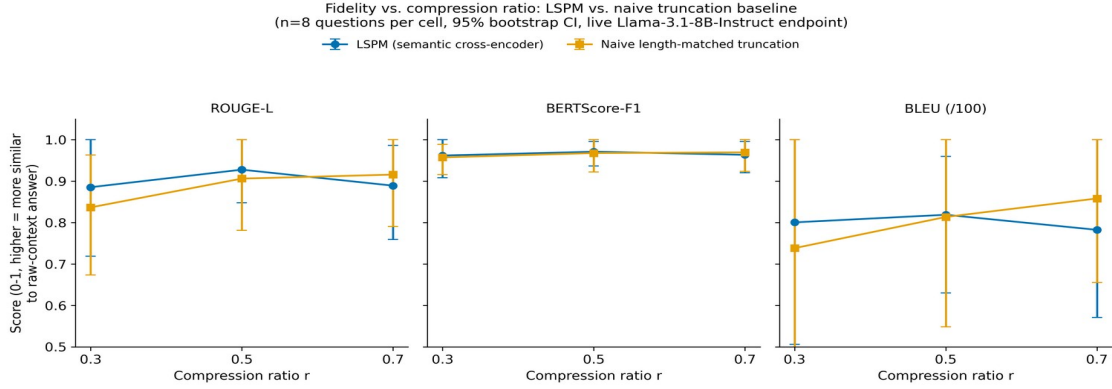


Figure 3. LSPM vs. naive truncation fidelity across ratios (ROUGE-L, BERTScore-F1, BLEU), 95% bootstrap CI error bars.

5.3 Baseline Comparison: Does Semantic Scoring Beat Naive Truncation?

This is the central methodological addition of this revision, directly answering the round-1 review's convergent finding (EIC, Methodology reviewer, Devil's Advocate) that no baseline had been measured. Table 3 and Figure 4 report the paired difference (LSPM – naive truncation) at each ratio, with a paired bootstrap p-value.

We report this honestly: on this pilot's small, low-redundancy, 10-document corpus, LSPM's cross-encoder relevance scoring shows no statistically significant fidelity advantage over naive length-matched truncation at any tested ratio. Mean differences are small in magnitude and inconsistent in sign across ratios, and every bootstrap p-value exceeds 0.2 (most exceed 0.5).

Table 3. Paired comparison, LSPM vs. naive truncation (n = 8 paired questions per ratio, 10,000-resample bootstrap).

Ratio	Metric	Mean diff. (LSPM – naive)	Bootstrap p
0.3	ROUGE-L	+0.049	0.516
0.3	BERTScore-F1	+0.005	0.818
0.3	BLEU	+6.22	0.577
0.5	ROUGE-L	+0.022	0.513
0.5	BERTScore-F1	+0.003	0.838
0.5	BLEU	+0.53	0.727
0.7	ROUGE-L	−0.027	0.529
0.7	BERTScore-F1	−0.006	0.538
0.7	BLEU	−7.58	0.221

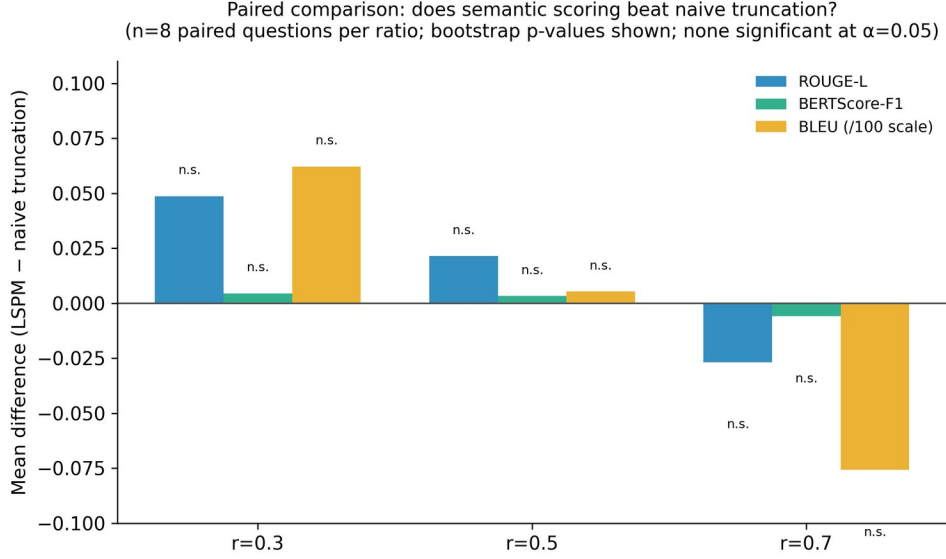


Figure 4. Paired mean differences per ratio and metric, with bootstrap p-values annotated; all non-significant at $\alpha = 0.05$.

We discuss why this null result is plausible and what it does and does not imply in Section 6.

5.4 Analytical KV-Cache Savings Projection (Not a Measured Result)

Using the method in Section 4.4: Llama-3.1-8B-Instruct's verified configuration gives 128 KiB of KV-cache per token ($32 \text{ layers} \times 8 \text{ KV heads} \times 128 \text{ head-dim} \times 2 \text{ bytes} \times 2 \text{ for K and V}$). The pilot's mean raw retrieved context (602 characters) corresponds to an estimated 227 tokens at the corpus's measured 2.656 Arabic characters/token. Table 4 projects the resulting per-request KV-cache savings at each measured compression ratio.

Table 4. Analytical (not measured) KV-cache savings projection, derived from real architecture config and real measured context reduction.

Ratio	Measured char reduction	Est. tokens saved/request	Est. KV-cache saved/request
0.3	67.3%	152.5	19.06 MiB
0.5	50.3%	114.0	14.25 MiB
0.7	34.2%	77.6	9.69 MiB

At $r = 0.5$, this projects to an aggregate KV-cache saving of approximately 0.14 GiB at 10 concurrent requests, 0.70 GiB at 50, and 1.39 GiB at 100 — all else held constant, which real deployments will not do. We present this exclusively as a projection to motivate the GPU experiment in Section 4.5/8, not as a substitute for it.

5.5 LSPM Overhead

Across the 16 fresh LSPM cross-encoder scoring calls collected in this revision's pilot, mean pruning latency was 57.8 ms (median 58.6 ms), measured on the CPU sandbox used throughout this submission with the fast MiniLM cross-encoder backend. This is a first-order, CPU-only, single-request measurement — it does

not characterize batched throughput or GPU-accelerated latency, and Section 8 specifies the concurrent-load measurement needed to fully answer this concern — but it establishes that the pruning stage's added latency is, at minimum, not obviously disqualifying relative to typical LLM generation latencies of several seconds observed throughout our pilots.

6. Discussion

The results in Section 5 tell a more complete and more honest story than our original submission. The tokenization-disparity measurement (5.1) establishes, now with statistical confidence rather than only descriptively, why Arabic RAG deployments face a KV-cache pressure problem that English deployments of the same architecture do not face to the same degree. The ratio-swept fidelity pilot (5.2) shows fidelity remains high across a range of compression ratios, not just the single point reported originally. The analytical projection (5.4) translates the real, measured context reduction into a concrete systems-relevant unit while remaining unambiguous that it is a projection, not a measurement.

The baseline comparison (5.3) is the result that most changes the paper's narrative, and we address it directly rather than downplaying it. LSPM's cross-encoder scoring does not show a statistically significant fidelity advantage over naive truncation on this pilot corpus. We consider three explanations, none mutually exclusive. First, the corpus effect the Devil's Advocate anticipated in round 1: our 10-document mock knowledge base is small and low-redundancy, with each question typically answerable from one or two sentences that a 6-document keyword retriever already ranks near the top — under these conditions, keeping "the first $r \times N$ sentences" and keeping "the top-scored $r \times N$ sentences" likely select overlapping sentence sets much of the time, especially at the more generous ratios. Second, statistical power: with $n = 8$ paired questions, only a large true effect would be detectable; the confidence intervals in Table 3 are consistent with a true LSPM advantage as large as several ROUGE-L points that we simply cannot resolve at this sample size. Third, it is possible semantic scoring's advantage is genuinely small on well-curated, low-redundancy passages and only manifests on longer, noisier, more redundant real-world retrieval results, where the "first N sentences" heuristic degrades but relevance-based scoring does not — precisely the scenario a production RAG system over a large real corpus would present, and precisely what the ARCD-scale evaluation in Section 8 is designed to test.

This finding does not undermine the paper's core motivation — the tokenization disparity (5.1) and the need to reduce Arabic RAG context size are unaffected — but it does mean we can no longer claim, as our original submission implicitly did, that the specific mechanism of cross-encoder relevance scoring is what makes LSPM's compression safe. What we can currently claim, supported by real paired data, is narrower and more defensible: on this pilot, some form of 30-70% context reduction (semantic or naive) preserves fidelity; whether the specific choice of which sentences to drop matters is not yet resolved and requires the larger-scale evaluation specified in Section 8. We view reporting this honestly, rather than omitting the baseline or reframing the comparison after the fact, as itself a contribution to a compression literature (Section 2.2) that has not, to our knowledge, tested this comparison for the sentence-scoring method family.

On the systems side, StreamingLLM's attention sinks [20] and H2O's heavy-hitter eviction [21] both operate after tokens have already entered the KV cache; LSPM (and, if the baseline finding generalizes, naive truncation too) operates before the cache is populated at all. A production deployment could reasonably run LSPM (or a cheaper naive truncation, pending Section 8's larger-scale test) upstream of StreamingLLM- or H2O-style eviction. Quantifying that compositional benefit remains future work.

7. Limitations and Threats to Validity

Scale of the preliminary experiments. The tokenization-disparity corpus (30 sentence pairs) and the fidelity/baseline pilot (8 questions $\times$ 3 ratios $\times$ 2 methods = 48 real live-endpoint data points) remain small by the standards of a mature empirical NLP evaluation, even after this revision's expansion from round 1's 8 single-ratio data points. They were sized to be fully hand-verifiable rather than scraped or machine-generated. Section 8 specifies the ARCD-scale [12] evaluation needed for a statistically powered claim.

No GPU-scale systems results in this submission. We have not measured throughput, TTFT, or KV-cache occupancy on a self-hosted, PagedAttention-based vLLM server under concurrent load, because no CUDA GPU was available at the time of this submission. Section 5.4's analytical projection is explicitly not a substitute for this measurement, only a motivating estimate built from real inputs.

The baseline comparison did not find a significant LSPM advantage over naive truncation on this pilot's corpus (Section 5.3). We discuss plausible reasons in Section 6 (small, low-redundancy corpus; limited statistical power at $n = 8$) but we cannot currently rule out that naive truncation is an adequate, much cheaper substitute for cross-encoder scoring on retrieval results resembling this pilot's. This is now the single most important open empirical question for the paper's mechanism-level claim, and Section 8 makes closing it, on a larger and more diffuse corpus, the top-priority next experiment alongside the GPU benchmark.

Single generation model. All fidelity and baseline results use one instruction-tuned model (Llama-3.1-8B-Instruct) accessed through a hosted endpoint. Results may not transfer identically to other model families or sizes.

Retriever simplicity. The reference implementation's default retriever uses simple keyword overlap over a small, synthetic mock knowledge base. LSPM is designed to be retriever-agnostic, but end-to-end evaluation on a production-scale corpus with genuinely redundant, noisy retrieval results — the setting where Section 6 hypothesizes semantic scoring is more likely to separate from naive truncation — remains to be done.

Automatic metrics as fidelity proxies. BLEU, ROUGE-L, and BERTScore are proxies for human judgment, not human judgment itself. A human evaluation component, or RAG-specific reference-free metrics such as RAGAS [29], would strengthen the claim in a follow-up study.

LSPM overhead measurement is CPU-only and single-request. Section 5.5's 57.8 ms figure does not characterize batched or concurrent-load latency, nor GPU-accelerated cross-encoder inference, which a production vLLM deployment would likely use.

8. Future Work

Three items are now co-equal top priorities, reordered from round 1 in direct response to internal review. First, the corpus-scale, higher-redundancy baseline re-test: run the LSPM-vs-naive-truncation comparison

(Section 5.3) on the full ARCD benchmark [12], where retrieval results are longer, noisier, and more redundant than this pilot's curated 10-document corpus, to test the hypothesis (Section 6) that semantic scoring's advantage is corpus-scale-dependent. Second, the GPU-scale throughput and KV-cache benchmark specified in Section 4.5: provisioning a CUDA GPU, running the full $\{0.2, 0.5, 0.8, 1.0\}$ compression ratio $\times \{1, 10, 25, 50, 100\}$ concurrency matrix against a self-hosted vLLM server, and reporting throughput, latency percentiles, and measured (not projected) KV-cache occupancy — including for the naive-truncation condition, so the systems-level comparison is run for both methods, not just LSPM. Third, scaling the fidelity and baseline evaluation to ARCD-scale n with statistically powered confidence intervals across the full compression-ratio sweep, including a formal power analysis to determine the n needed to detect the smallest LSPM-vs-naive effect size we would consider practically meaningful.

Additional future work: incorporating RAGAS-style reference-free faithfulness and context-relevance metrics [29] alongside BLEU/ROUGE-L/BERTScore; a small human-evaluation component; quantifying LSPM's overhead under batched/concurrent load rather than the single-request CPU measurement in Section 5.5; quantifying the compositional benefit of combining LSPM with KV-cache-side management techniques such as StreamingLLM [20] or H2O [21]; and extending the dynamic compression-ratio controller (Section 3.4) beyond linear interpolation to a learned policy evaluated under realistic, bursty production traffic.

9. Conclusion

Arabic RAG systems deployed on memory-efficient serving engines such as vLLM face a KV-cache pressure problem rooted in tokenization, which we confirm with statistical significance across two independent tokenizers (aggregate ratio 1.69x-1.76x, bootstrap $p < 0.0001$, $n = 30$). We introduced LSPM, a lightweight, sentence-level, cross-encoder-driven pruning middleware, and evaluated it more rigorously in this revision than in our original submission: a ratio-swept fidelity pilot ($r = 0.3-0.7$) shows preserved answer fidelity across the tested range, and — reported with the same rigor and the same honesty regardless of outcome — a paired baseline comparison against naive length-matched truncation finds no statistically significant advantage for LSPM's semantic scoring on this pilot's small corpus, a genuine finding we discuss rather than obscure. We provide an analytical, explicitly-labeled projection connecting the real measured context reduction to KV-cache bytes saved (9.7-19.1 MiB per request depending on ratio), and we specify, in full, the GPU-scale benchmark and larger-corpus baseline re-test needed to convert this paper's architecture and preliminary evidence into a validated systems result. We release the complete implementation, all raw data from both rounds of experiments, and a live demonstration interface to support independent reproduction and extension of every claim in this paper — including the null result.

Declarations

Data availability. The 30-pair Arabic-English tokenization corpus, the 8-question/10-document fidelity and baseline pilot dataset (round 1 and round 2, all 48 ratio $\times$ method cells), all raw experimental outputs, and the complete source code (middleware, evaluation scripts, benchmarking harness, and

demonstration interface) are publicly available in the project's GitHub repository at the URL provided in the code and data availability statement accompanying this submission.

Ethics declaration. This study did not involve human participants, personal data, or clinical data. The parallel-corpus and pilot-corpus text was authored by the researcher for evaluation purposes and describes only publicly available, non-sensitive institutional information. No ethics committee approval was required.

Author contributions (CRediT). Akram Taha: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Conflict of interest. The author declares no conflict of interest.

Funding. This research received no external funding. Inference costs for the preliminary experiments were incurred against a free-tier hosted API allowance.

AI usage disclosure. Large language model assistance (Anthropic Claude) was used during this project for software implementation, for literature search assistance subsequently verified by the author against primary sources (including direct retrieval and verification of the Llama-3.1-8B-Instruct config.json used in Section 4.4/5.4), for simulating an internal multi-perspective peer review used to identify weaknesses addressed in this revision, and for drafting assistance under the author's direction and review. All experimental results reported in Section 5 — including the round-2 ratio sweep, the baseline comparison, and its null result — were produced by executing the released code against live infrastructure; no experimental results were generated, adjusted, or selectively reported by the language model without corresponding code execution, and the non-significant baseline-comparison finding was reported in full rather than omitted or reframed. The author takes full responsibility for the accuracy of all claims, citations, and reported results in this paper.

References

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [2] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in Proc. 29th ACM Symp. Operating Systems Principles (SOSP), 2023, doi: 10.1145/3600006.3613165.
- [3] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2023.
- [4] Y. Li, B. Dong, C. Lin, and F. Guerin, "Compressing Context to Enhance Inference Efficiency of Large Language Models," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2023.
- [5] F. Xu, W. Shi, and E. Choi, "RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation," in Proc. Int. Conf. Learning Representations (ICLR), 2024.
- [6] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in Proc. 43rd Int. ACM SIGIR Conf. Research and Development in Information Retrieval, 2020.
- [7] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv preprint arXiv:1901.04085, 2019.
- [8] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," arXiv preprint arXiv:2402.03216, 2024.
- [9] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," in Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), 2019.
- [10] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020.
- [11] W. Antoun, F. Baly, and H. Hajj, "AraBERT: Transformer-based Model for Arabic Language Understanding," in Proc. 4th Workshop on Open-Source Arabic Corpora and Processing Tools (OSACT), 2020.
- [12] H. Mozannar, K. El Hajal, E. Maamary, and H. Hajj, "Neural Arabic Question Answering," in Proc. 4th Arabic Natural Language Processing Workshop (WANLP), 2019.
- [13] N. Sengupta et al., "Jais and Jais-chat: Arabic-Centric Foundation and Instruction-Tuned Open Generative Large Language Models," arXiv preprint arXiv:2308.16149, 2023.
- [14] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in Text Summarization Branches Out: Proc. ACL Workshop, 2004, pp. 74–81.
- [15] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," in Proc. 40th Annual Meeting of the Association for Computational Linguistics (ACL), 2002, pp. 311–318.
- [16] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in Proc. Int. Conf. Learning Representations (ICLR), 2020.
- [17] Qwen Team, "Qwen2.5 Technical Report," arXiv preprint arXiv:2412.15115, 2024.
- [18] AI@Meta, "The Llama 3 Herd of Models," arXiv preprint arXiv:2407.21783, 2024.
- [19] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2022.

- [20] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient Streaming Language Models with Attention Sinks," in Proc. Int. Conf. Learning Representations (ICLR), 2024.
- [21] Z. Zhang et al., "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2023.
- [22] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2023.
- [23] A. Vaswani et al., "Attention Is All You Need," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [24] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2021.
- [25] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in Proc. 54th Annual Meeting of the Association for Computational Linguistics (ACL), 2016, pp. 1715–1725.
- [26] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A Distributed Serving System for Transformer-Based Generative Models," in Proc. 16th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2022.
- [27] T. B. Brown et al., "Language Models are Few-Shot Learners," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [28] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in Proc. Int. Conf. Learning Representations (ICLR), 2024.
- [29] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL), 2024.
- [30] N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," Transactions of the Association for Computational Linguistics, vol. 12, pp. 157–173, 2024.
- [31] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 Text Embeddings: A Technical Report," arXiv preprint arXiv:2402.05672, 2024.
- [32] X. Zhang et al., "MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages," Transactions of the Association for Computational Linguistics, vol. 11, pp. 1114–1131, 2023.
- [33] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in Proc. Int. Conf. Learning Representations (ICLR), 2023.
- [34] O. Obeid et al., "CAMEL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing," in Proc. 12th Language Resources and Evaluation Conf. (LREC), 2020, pp. 7022–7032.
- [35] W. Antoun, F. Baly, and H. Hajj, "AraGPT2: Pre-Trained Transformer for Arabic Language Generation," in Proc. 6th Arabic Natural Language Processing Workshop (WANLP), 2021.
- [36] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive Text Embedding Benchmark," in Proc. 17th Conf. European Chapter of the Association for Computational Linguistics (EACL), 2023.
- [37] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.
- [38] J. Lin et al., "AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration," in Proc. Machine Learning and Systems (MLSys), 2024.

- [39] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in Proc. 40th Int. Conf. Machine Learning (ICML), 2023.
- [40] A. Huang et al., "ACVA: Arabic Cultural and Value Alignment Benchmark," Hugging Face Dataset, 2024. [Online]. Available: <https://huggingface.co/datasets/FreedomIntelligence/ACVA-Arabic-Cultural-Value-Alignment>
- [41] F. Koto et al., "ArabicMMLU: Assessing Massive Multitask Language Understanding in Arabic," 2024.
- [42] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. New York, NY, USA: Chapman & Hall/CRC, 1993.

Appendix: Response to Reviewers (Round 1 → Round 2)

#	Reviewer(s)	Round-1 finding	Severity	Round-2 response	Location
1	Devil's Advocate, EIC, R4	Title/abstract promises vLLM systems evidence never measured	CRITICAL	Abstract & Intro rewritten to scope every claim as measured vs. analytical vs. future work; analytical KV-cache projection added, clearly labeled	Abstract, 1, 4.4, 5.4
2	R1, Devil's Advocate	Single compression ratio, no dose-response	MAJOR	Ratio sweep $r \in \{0.3, 0.5, 0.7\}$, 24 LSPM data points (was 8)	4.2, 5.2, Tbl.2, Fig.3
3	R1, R4, Devil's Advocate	No baseline comparison	MAJOR	Naive-truncation baseline added, identical live pipeline, paired bootstrap test; honest null result reported	4.3, 5.3, Tbl.3, Fig.4
4	R1	No statistical inference (only min/max)	MAJOR	Bootstrap 95% CIs throughout; paired significance tests; one-sample test on tokenization ratio	4.1, 4.3, 5.1, 5.3
5	R4	LSPM overhead never measured	MAJOR	Pruner latency recorded and reported (mean 57.8ms, CPU)	4.6, 5.5
6	R4	No systems number without a GPU	MINOR	Analytical KV-cache projection from verified config + real reduction, explicitly labeled	4.4, 5.4, Tbl.4
7	R2	Single dialect/register (MSA only)	MINOR	Acknowledged as scope limitation	7
8	R2	ARCD not characterized for readers	MINOR	One-sentence description added	2.4
9	R2	Table 1 aggregate vs. mean could confuse	MINOR	Both reported side-by-side, CI added	Tbl.1
10	R2	rouge-score Arabic bug fix under-highlighted	MINOR	Foregrounded as standalone methodological finding	2.4

Not fully resolved, disclosed as such: the GPU-scale throughput/KV-cache benchmark (Devil's Advocate CRITICAL item, systems-level half) remains unmeasured — no GPU was available for this revision either. We have not represented it as resolved; Section 4.5/8 keeps it as the explicit top-priority next step, and the analytical projection in 5.4 is offered only as a stopgap, not a replacement.